Lab Assignment 7: The MAC Sublayer

Computer Networks Lab - IIIT Guwahati

Prerequisites and tools

- Lecture slides: MAC Layer, Part 2 (Fast Ethernet through 802.1Q).
- Python 3.10 or later with `matplotlib`. No other libraries are needed; do not use `scapy` or any networking package for the graded code.
- Wireshark, to check your Part C decoder against a real capture.
- Create one folder `lab4_<rollno>/` with one file per part: `partA.py` … `partE.py`, plus `report.pdf`.

Fast Ethernet keeps every MAC rule and only shrinks the bit time from 100 ns to 10 ns; this part you need to compute that shrink and then build the 100Base-TX and 100Base-FX encoders that achieve it.

802.11 MAC header decoder

You need to decode six captured 802.11 headers, first by hand and then with a Python parser, and use the ToDS/FromDS bits to name every address.

Format reminders

- Frame Control is 2 bytes. In the first byte, bits 0–1 are Protocol Version, bits 2–3 are Type (00 management, 01 control, 10 data) and bits 4–7 are Subtype. The second byte holds the flags, from bit 0 upward: ToDS, FromDS, More Frag, Retry, Power Mgmt, More Data, Protected, Order.
- Duration/ID and Sequence Control are 16-bit **little-endian** fields. In Sequence Control, the low 4 bits are the fragment number and the high 12 bits are the sequence number.
- Address 4, when present, comes **after** Sequence Control.
- Control subtypes you need: 1011 = RTS, 1100 = CTS, 1101 = ACK. RTS carries two addresses; CTS and ACK carry only one.

Captured frames (header bytes only, hex)

```
F1: 08 01 2c 00 00 1a 2b 3c 4d 5e a4 83 e7 11 22 33
    00 50 56 aa bb cc 30 1d aa aa 03 00 00 00 08 00
F2: 08 0a 2c 00 a4 83 e7 11 22 33 00 1a 2b 3c 4d 5e
    00 50 56 aa bb cc 40 1d aa aa 03 00 00 00 08 00
F3: b4 00 5e 01 00 1a 2b 3c 4d 5e a4 83 e7 11 22 33
F4: c4 00 44 01 a4 83 e7 11 22 33
F5: d4 00 00 00 a4 83 e7 11 22 33
F6: 08 07 2c 00 02 11 22 33 44 55 02 66 77 88 99 aa
    00 50 56 aa bb cc 51 1d a4 83 e7 11 22 33 aa aa
    03 00 00 00 08 00
```

1. Decode by hand (5 marks)

For F1 and F3, fill this table in your report. Show the binary of the Frame Control bytes.

Field	F1	F3
Type / Subtype		
ToDS, FromDS		
Other flags set		
Duration (µs)		
Address 1 (role)		
Address 2 (role)		

Address 3 (role)		
Sequence no. / fragment no.		

2. Parser (7 marks)

Write Python code (`parser.py`), write `decode_80211(hexstr)` that returns a dictionary with type, subtype name, all flags, duration, every address labelled with its role (RA, TA, DA, SA or BSSID, following the ToDS/FromDS table on the slides), sequence and fragment numbers, and the EtherType from the LLC/SNAP header when the frame is data. Print a neat one-block summary for each of F1–F6.

Your code must pick the right address roles purely from the ToDS/FromDS bits, handle frames with one, two, three or four addresses, and report an error for input that is too short.

C3. Interpretation (3 marks)

1. F3, F4 and F5 are one exchange. Name the sender and receiver of each, and explain why the Duration drops from F3 to F4 to F5.
2. A station that hears only F4 sets its NAV. For how many microseconds, and which slide mechanism does this implement?
3. F2 has the Retry flag set. How does the receiver use the Retry bit together with the sequence number to discard duplicates?
4. Why does F6 need a fourth address? Sketch the path it takes, using the WDS figure from the slides.

Optional check: capture your own Wi-Fi traffic in monitor mode with Wireshark and compare one data frame against your parser's output.